

Aufgabe 3: Lieferkette

Teilnahme-ID: 78920

Bearbeiter/-innen dieser Aufgabe:

Tao Zheng

2. September 2026

Inhaltsverzeichnis

1	Lösungsidee	2
1.1	Graphenbildung	2
1.2	DP Ansatz	2
1.2.1	Der direkte kürzeste Weg (Fall 0)	2
1.2.2	Der kürzeste Weg über einen Alternativfabrik (Fall 1)	3
1.2.3	Der optimale Weg (Fall 2 und 3)	3
1.3	Dial's Algorithmus	3
1.4	Ergebnisausgabe	4
2	Umsetzung	4
2.1	Durchführung der Dial's Algorithmus für den Fall 0 und 1	5
2.2	Durchführung der Dial's Algorithmus für den Fall 2 und 3	6
2.3	Endergebnis	7
3	Analyse	7
3.1	Laufzeit- und Speicheranalyse	7
3.2	Fragestellung Komplexitätseinstufung	8
4	Werkzeuge	8
5	Beispiele	8
5.1	Vorgegebene Aufgaben	9
5.2	Eigene Aufgaben	10
6	Quellcode	12
7	Zusatzherausforderung: Bitmask Lieferprobleme	15
8	Zusatzherausforderung: Lösungsidee	15
8.1	Graphenbildung	15
8.2	Dynamische Programmierung	16
8.2.1	Der direkte Weg (Fall 0)	16
8.2.2	Der kürzeste Weg über ein Alternativwerk (Fall 1)	16
8.2.3	Der optimale Weg (Fall 2 und 3)	16
8.3	Backtracking	17
8.4	Längenberechnungen	17
9	Zusatzherausforderung: Umsetzung	17
10	Zusatzherausforderung: Analyse	19
10.1	Laufzeit und Speicherplatzkomplexität	19
10.1.1	Dial's Algorithmus	19
10.1.2	DP mit Bitmask	19
10.1.3	Ergebnis	20
10.2	Problemeinstufung	20
11	Zusatzherausforderung: Beispiele	21
11.1	Vorgegebene Aufgaben	21
11.2	Eigene Aufgaben	23
12	Zusatzherausforderung: Quellcode	25

1 Lösungsidee

Das gegebene Problem kann mithilfe eines Programms gelöst werden, das die Worst-Case Weglänge minimiert. Die Lösung konzentriert sich daher auf die Entwicklung eines Programms, das den minimalen Worst-Case-Weg (auch optimaler Weg genannt) berechnet und die entsprechende Wegbeschreibung ausgibt.

1.1 Graphenbildung

Wir bilden den ungerichteten, gewichteten Graphen $G = (V, E)$ mit den Gewichten $w : E \rightarrow \{1, 2, \dots, 20\}$. Zusätzlich bilden wir die Menge $A = \{0, 1, \dots, k, k+1\}$ mit $a : V \rightarrow A$, welche die auszuführenden Arbeitsschritte darstellt, wobei k für die Gesamtzahl der Arbeitsschritte steht und zugleich folgende Eigenschaften gelten:

- $a^{-1}(0) = \{S\}$ (Startknoten)¹
- $a^{-1}(k+1) = \{T\}$ (Zielknoten)
- $a^{-1}(i) \subset V \forall i \in \{1, 2, \dots, k\}$, die Menge der Werkstätten ist, die die Aufgabe i erfüllen können.

Diese Graphenbildung brauchen wir, um später mithilfe des Dial's Algorithmus² Multi-Source Shortest Path Kalkulationen – unter Verwendung des Graphen G sowie der Gewichtsfunktion w – durchzuführen. Der Dial's Algorithmus wird k mal ausgeführt (für jeden Arbeitsschritt), wobei alle Knoten aus dem jeweiligen Arbeitsschritt als Startpunkt ausgewählt werden. Da der kürzeste Weg abhängig von der Distanz aller Knoten zum Endknoten T unter Beachtung der einzelnen Arbeitsschritte ist, werden die Ergebnisse aus vorherigen Berechnungen als Startwert der Startpunkte übernommen. Ich werde später darauf tiefer eingehen, da dafür der DP-Ansatz zuerst noch erläutert werden muss, aber für jetzt ist erst einmal anzunehmen, dass mithilfe des Dial-Algorithmus die Funktion $\text{dist}(u, v)$ existiert, da diese für das Verstehen des DP-Teils erforderlich sein wird. Da aber die Funktion $\text{dist}(u, v)$ nicht im Zusammenhang mit der Multi-Source Idee funktionieren kann, wird später ebenfalls genauer erläutert, wie diese Funktion tatsächlich angewendet wurde.

- Annahme: Es gibt eine Funktion $\text{dist}(u, v)$, $u \in V, v \in V$, welche mithilfe des Graphen G sowie der Gewichtsfunktion w die kürzeste Weglänge zwischen zwei Knoten zurückgibt.

Zusatz:

- Aus Gründen der Orientierung habe ich mir vorgenommen, dass das Wort „Anfangsknoten“ im Artikel dem im Dials Algorithmus gesetzten Startknoten entspricht.

1.2 DP Ansatz

Der DP-Ansatz wird für den späteren Dial-Algorithmus vorbereitet. Während der Berechnung im Dial-Algorithmus wird der Endknoten T als Startpunkt ausgewählt. Dadurch stellen alle Werte die Distanz zwischen dem aktuellen Knoten $u \in V$ zum Endknoten T dar, wodurch folgende Frage gestellt werden kann: „Wie weit bin ich vom aktuellen Knoten u zum Endknoten T entfernt?“ (Später: Fall 0). Diese Frage kann anschließend zu der Frage: „Wie weit bin ich vom aktuellen Knoten u zum Endknoten T entfernt, wenn ich zuerst zu einem Alternativweg $v \in V$ gehen würde?“ (Später: Fall 1) abgeleitet werden. Hätte man vom Startknoten S aus begonnen, wäre eine solche Ableitung auf eine effiziente Weise nicht möglich gewesen.

Wir erstellen daher den $\text{DP} : V \times z \rightarrow \mathbb{N}_0$, wobei $u \in V$ und $z \in \{0, 1, 2, 3\}$ gilt.

Für die Initialisierung gilt:

$$\text{DP}(u, z) = \begin{cases} 0 & \text{falls } u = T \\ \infty & \text{falls } u \in V \setminus \{T\} \end{cases}$$

1.2.1 Der direkte kürzeste Weg (Fall 0)

Fall 0 ($z = 0$) berechnet den kürzesten direkten Weg vom Knoten u zum Endknoten T unter der Beachtung von noch auszuführenden Arbeitsschritten. Für die Berechnung gilt folgende Formel:

¹vgl. [https://de.wikipedia.org/wiki/Urbild_\(Mathematik\)](https://de.wikipedia.org/wiki/Urbild_(Mathematik))

²Dieser Algorithmus ist für diesen Fall geeignet, da alle Kanten ein Höchstgewicht von 20 besitzen. Die Laufzeitkomplexität für einen Durchlauf beträgt $\mathcal{O}(N \cdot w + E)$, wobei w den Wert 20 besitzt. Sind die Gewichte deutlich größer als 20, so muss man den Dial-Algorithmus durch Dijkstra ersetzen lassen.

$$DP(u, 0) = \min_{a(u)+1=a(v)} (\text{dist}(u, v) + DP(v, 0))$$

1.2.2 Der kürzeste Weg über einen Alternativfabrik (Fall 1)

Fall 1 ($z = 1$) berechnet den kürzesten Weg von Knoten u zum Endknoten T über ein Alternativwerk v (es gilt: $a(v) = a(u)$), ebenfalls unter der Beachtung von noch auszuführenden Arbeitsschritten. Das heißt, man möchte für alle Knoten u einen anderen Knoten v aus der gleichen Arbeitsschritt-Gruppe finden, um im Falle eines Ausfalls dorthin zu gehen ($\text{dist}(u, v)$) und danach davon auszugehen, dass keine weiteren Fabriken streiken werden ($DP[v][0]$). Für die Berechnung gilt daher folgende Formel:

$$DP(u, 1) = \min_{a(u)=a(v)} (\text{dist}(u, v) + DP(v, 0))$$

Hinzu gilt immer:

$$DP(u, 0), DP(v, 0) \leq DP(u, 1)$$

Um diese Formel in der Multi-Source Berechnung verwenden zu können, muss die Formel noch umgewandelt werden (da alle Knoten aus dem gleichen Arbeitsschritt im Dial-Algorithmus als Anfangsknoten ausgewählt werden). Dies wird später ebenfalls genauer erläutert.

1.2.3 Der optimale Weg (Fall 2 und 3)

Fall 2 ($z = 2$) berechnet den Weg mit der minimalen Worst-Case Wegdauer von Knoten u zum Endknoten T unter der Beachtung von noch auszuführenden Arbeitsschritten. Dabei gilt folgende Formel:

$$DP(u, 2) = \max \left(\min_{a(u)+1=a(v)} (\text{dist}(u, v) + DP(v, 2)), DP(u, 1) \right)$$

Der Worst-Case Weg entspricht somit dem längsten Weg von allen vorherigen Worst-Case Wegen, die der Knoten verwendet, oder dem aktuellen Ersatzwerk Weg. Dabei wird darauf geachtet, dass alle vorherigen Worst-Case Wege minimiert werden. Da aus Fall 1 folgende Gleichung immer gilt: $DP(u, 0) \leq DP(u, 1)$, muss nur Fall 1 überprüft werden.

Fall 3 berechnet die tatsächliche Länge des Weges, welcher in Fall 2 berechnet wurde. Dabei gilt:

$$DP(u, 3) = \text{dist}(u, v) + DP(v, 3)$$

Für die Formel gilt, dass u und v die gleichen Belegungen haben müssen wie im Fall 2.

1.3 Dial's Algorithmus

Wie schon zuvor angedeutet, wird das Multi-Source Verfahren mit übernommenen Ergebnissen angewendet. Es ist wichtig, dass das Verfahren zuerst für die Fälle 0 und 1 vollständig durchgerechnet werden muss, bis die Fälle 2 und 3 bearbeitet werden. Bei jedem Verfahren wird für $\text{task} = k + 1$ (Endknoten T) angefangen und alle Werte für task absteigend berechnet. Alle Anfangsknoten u erhalten als Startgewicht $DP(u, 0)$, um alle Knoten von der Distanz zum Endknoten T abhängig zu machen. Während des Verfahrens wird ein Backtracking eingebaut, das für alle Knoten auf deren Vorgängerknoten zeigt (wobei der Vorgänger eines Anfangsknotens die Anfangsknoten selbst ist), um die Weganweisungen später iterativ wiederzugeben. Wenn eine Verbindung zu einem Knoten u mit der Eigenschaft $a(u) = \text{task} - 1$ gefunden wird, so wird überprüft, ob dieser Weg kürzer als $DP(u, 0)$ ist. Das liegt daran, dass das Startgewicht eines Anfangsknotens $DP(v, 0)$ ist und durch das Verfahren mathematisch gesehen: $DP(v, 0) + \text{dist}(u, v)$ berechnet wurde, welches der Formel aus Fall 0 entspricht. Diese Idee kann ebenfalls für Fall 2 und 3 verwendet werden (indem man für $z = 2$ einsetzt), wobei man nach jedem Arbeitsschritt für alle Knoten aus dem nächsten Arbeitsschritt schauen muss, ob Fall 1 größer als der berechnete Weg ist (vgl. Fall 2 Formel).

Die Berechnung von Fall 1 wird komplexer. Das liegt daran, dass im Algorithmus alle Knoten aus einem Arbeitsschritt als Anfangsknoten gesetzt werden, weshalb es nicht wirklich möglich ist, einzelne Distanzen zu berechnen. Unter der Verwendung von mindestens zwei Anfangsknoten werden sich die Knoten in der Mitte treffen, wodurch sich die Länge von beiden Anfangsknoten zum Mittelpunkt berechnen kann. Aus diesem Grund wird für jeden Arbeitsschritt die Funktion $\text{captured} : V \rightarrow V$ erstellt, die den

Anfangsknoten u eines Knotens v zeigt, der den kürzesten Weg möglich macht (wobei $\text{captured}(u) = u$ für den Anfangsknoten u gilt). Während der Berechnung von Fall 0 wird daher für jeden erreichten Knoten der Verweis auf den Anfangsknoten erstellt.

Die Formel für den Fall kann daher von:

$$\text{DP}(u, 1) = \min_{a(u)=a(v)} (\text{dist}(u, v) + \text{DP}(v, 0))$$

für den Fall, dass u und v jetzt die zwei in der Mitte treffenden Knoten aus verschiedenen Startknoten sind, angepasst werden:

$$\begin{aligned} &\text{DP}(\text{captured}(u), 1) = \\ &= \min_{a(\text{captured}(u))=a(\text{captured}(v))} (\text{dist}(\text{captured}(u), u) + \text{dist}(u, v) + \text{dist}(v, \text{captured}(v)) + \text{DP}(\text{captured}(v), 0)) \end{aligned}$$

Dadurch können die Fälle 0, 1, 2 und 3 vollständig bearbeitet werden.

1.4 Ergebnisausgabe

Der Wert $\text{DP}(S, 2)$ entspricht der absoluten Worst-Case Zeit, welche mit dem in der Problemstellung gegebenen Zeitlimit zu vergleichen ist. Die Länge der empfohlenen Route entspricht $\text{DP}(S, 3)$. Um die Gesamtlänge eines Ersatzweges zu berechnen, wird folgende Formel verwendet:

$$\text{Länge} = \text{DP}(S, 3) - \text{DP}(u, 3) + \text{DP}(u, 1)$$

wobei der Knoten u für das streikende Werk aus einem der im empfohlenen Weg genannten Werke steht. Um die Wegbeschreibung auszugeben, werden die einzelnen Werte aus dem gespeicherten Backtracking Objekt ausgegeben.

2 Umsetzung

Das Programm wurde in C++ umgesetzt. Im Folgenden ist eine Liste von eigenen Variablen mit deren Bedeutung zu finden:

- `int timelimit`: Der maximale Worst-Case Dauer, basierend auf der Aufgabenstellung
- `int tasks`: Anzahl der Arbeitsschritte
- `int rev_ids`: Funktion $a(u)$ aus der Lösungsidee.
- `int ids`: Funktion $a^{-1}(u)$ aus der Lösungsidee.

Um den Graphen G zu erzeugen, werden folgende Variablen definiert:

- `vector<vector<pair<int, int>>> adj(N)`: Adjazenzliste für alle Knoten des Graphen G .
- `int N`: Anzahl der Knoten im Graphen G inklusive des Start- und des Endknotens.
 - Es gilt: Startknoten $S \rightarrow$ Knoten 0
 - Es gilt: Endknoten $T \rightarrow$ Knoten $N - 1$

Um Backtracking zu ermöglichen, werden folgende Listen erstellt:

- `vector<vector<int>> backtrack_Case0(tasks + 2, N)`: Gibt aus einem Knoten u den besten Nachfolger zurück, wodurch der beste Weg möglich ist unter der Annahme, dass keine weiteren Werke mehr streiken.
- `vector<vector<string>> backtrack_Case1(N)`: Speichert für jeden Knoten u einen alternativen Weg über einen Knoten v , welcher die gleichen Arbeitsschritte wie u durchführen kann.
- `vector<vector<int>> backtrack_Case2(tasks + 2, N)`: Gibt aus einer Knoten u den Nachfolger mit der minimalen Worst-Case Weglänge zurück.
- `vector<int> recommendedPaths(N)`: ähnlich wie `backtrack_Case2`, jedoch wird diesmal nur die nächste empfohlene Fabrik gespeichert. Die einzelnen Knoten, die dazwischen liegen, werden darin nicht gespeichert.

Der Dial's Algorithmus wird zwei Mal separat voneinander definiert, wobei beim ersten Durchgang die Fälle 0 und 1 bearbeitet werden, und beim zweiten Durchgang die Fälle 2 und 3. Dabei wurden die Variablen aus <https://www.geeksforgeeks.org/dsa/dials-algorithm-optimized-dijkstra-for->

small-range-weights/ übernommen, und zusätzlich noch einige Veränderungen durchgeführt. Für beide Durchgänge werden folgende Sachen definiert:

- `vector<vector<int>> DP(N, 4)`: Für jeden Knoten u werden im DP alle Ergebnisse aus Fall 0 bis 3 gespeichert. Es gelten die in der Lösungsidee genannten Prinzipien.
- `int maxDist`: Dieser Wert stellt die weiteste Distanz dar, die aus einem Anfangsknoten erreichbar ist. Da einige Anfangsknoten mit einem Gewicht starten, wurde die Kapazität zusätzlich um den Faktor 2 erweitert. Diese Variable wird für die Erstellung von `buckets` verwendet. (vgl. [geeksforgeeks.org](https://www.geeksforgeeks.org/))
- `int bias`: Führt man den Dial's Algorithmus für mehrere Knoten bei unterschiedlichen Arbeitsschritt-Fortschritten durch, so haben die Anfangsknoten alle schon bereits ein Gewicht. Mithilfe von `bias` wird der kleinste Anfangswert aller Anfangsknoten ermittelt, um in `buckets` alle Anfangsknoten um `bias` Positionen ins Negative zu verschieben. Dadurch können die Anfangsknoten - unabhängig von der Gewicht - alle in `buckets` reinpassen und behalten dabei die Gewichtsunterschiede untereinander, wodurch alle Längen korrekt berechnet werden können.
- `int highDist`: Dabei handelt es sich um eine Optimierung, um den weitesten, erreichbaren Wert zu ermitteln. Dadurch kann eine weitere Abbruchbedingung für die Dial's Algorithmus erstellt werden, weil man weiß, dass keine weiteren Knoten nach `highDist` sein können und die restlichen `buckets`-Speicherzellen daher nicht überprüft werden müssen.

2.1 Durchführung der Dial's Algorithmus für den Fall 0 und 1

Um Fall 1 zu berechnen, wird für den Anlauf noch folgende Variable definiert:

- `vector<vector<int>> captures(tasks + 2, N)`: Stellt die Funktion `captured` aus der Lösungsidee dar. Alle Knoten zeigen dadurch auf einen Anfangsknoten des Dial's Algorithmus, wodurch weitere Berechnungen - v.a. im Zusammenhang der Fall 1 - möglich gemacht werden können.
- `vector<vector<int>> dist(tasks + 2, N)`: Berechnet für jeden Durchlauf die Distanz eines Knotens u zu dem Endknoten T in der Abhängigkeit davon, welcher Arbeitsschritt als Nächstes gemacht werden muss.

Sobald alle Werte initialisiert wurden, kann der Dial's Algorithmus angewendet für die Berechnung von Fall 0 und 1 verwendet werden. Eine Schleife für `int task = tasks + 1` bearbeitet alle einzelnen Arbeitsschritte absteigend und setzt alle Knoten u aus $a^{-1}(\text{task})$ als Anfangsknoten mit deren Startgewicht aus `DP[u][0]`.

Für `bias` gilt folgende Formel:

$$\text{bias} = \min_{u \in a^{-1}(\text{task})} (\text{bias}, \text{DP}[u][0])$$

Dadurch können alle Knoten beim Einfügen in `buckets` um `bias` Werte verschoben werden und Zugriffe auf ungültige Speicherzellen (Out of Bounds Access) verhindert werden.

Während der Dial's Algorithmus werden die alle erreichte Knoten mit `captures[task]` auf ihren Anfangsknoten und mit `backtrack_Case0[task]` auf ihren Vorgängerknoten verwiesen.

Wird aus einem Knoten u eine Kante zu einem Knoten v aus der nächsten Arbeitsschritt-Gruppe erreicht, so wird `DP[v][0] = min(DP[v][0], dist[task][v])` verwendet, um den kürzeren Weg einzuspeichern. Dadurch wird auch Fall 0 vollständig bearbeitet.

Findet sich ein Knoten u ein anderer Knoten v , die auf unterschiedliche Anfangsknoten verweisen (`captures[task][u] != captures[task][v]`, `captures[task][v] != -1`), so kann es sich hierbei um einen möglichen kurzen Alternativweg für beide Anfangsknoten aus der gleichen Arbeitsschrittgruppe handeln. Wir wandeln daher die aus der Lösungsidee genannte Fall 1 Formel für Standardfälle von:

$$\text{DP}[u][1] = \min_{a(u)=a(v)} (\text{dist}(u, v) + \text{DP}[v][0])$$

mit der Ersetzung von $u = \text{captures}[\text{task}][u]$, $v = \text{captures}[\text{task}][v]$ zu:

$$\text{DP}[\text{captures}[\text{task}][u]][1] = \min_{a(\text{captures}[\text{task}][u])=a(\text{captures}[\text{task}][v])} (\text{dist}(\text{captures}[\text{task}][u], \text{captures}[\text{task}][v]) + \text{DP}[\text{captures}[\text{task}][v]][0])$$

gefolgt von mit

$$\begin{aligned} & \text{dist}(\text{captures}[\text{task}][u], \text{captures}[\text{task}][v]) = \\ & = \text{dist}[\text{task}][u] - \text{dist}[\text{task}][\text{captures}[\text{task}][u]] + \text{dist}(u, v) + \text{dist}[\text{task}][v] - \text{dist}[\text{task}][\text{captures}[\text{task}][v]] \end{aligned}$$

sowie

$$\text{DP}[\text{captures}[\text{task}][v]][0] = \text{dist}[\text{task}][\text{captures}[\text{task}][v]]$$

zu der Endformel:

$$\begin{aligned} & \text{DP}[\text{captures}[\text{task}][u]][1] = \\ & = \min_{a(\text{captures}[\text{task}][u])=a(\text{captures}[\text{task}][v])} (\text{dist}[\text{task}][u] - \text{dist}[\text{task}][\text{captures}[\text{task}][u]] + \text{dist}(u, v) + \text{dist}[\text{task}][v]) \end{aligned}$$

Der Wert von $\text{dist}(u, v)$ beträgt das direkte Kantengewicht zwischen den Knoten u und v . Zusätzlich gilt noch für diese Formel die Einschränkung, dass $\text{captures}[\text{task}][u] \neq \text{captures}[\text{task}][v]$, $\text{captures}[\text{task}][u] \neq -1$, $\text{captures}[\text{task}][v] \neq -1$ gelten muss, damit die Formel funktioniert. Diese Formel lässt sich ebenfalls anwenden, wenn man u und v vertauscht.

Im Programm wird daher für Fall 1 geschaut, ob es eine neuere, kürzere Strecke gibt. Im Falle einer kürzeren Strecke wird der neue Wert im DP gespeichert und anschließend sofort die Wegbeschreibung für `backtrack_Case1` berechnet. Für den Fall, dass der Anfangsknoten $\text{captures}[\text{task}][u]$ einen günstigeren Alternativ-Werk gefunden hat, werden zunächst einmal alle einzelnen Schritte von dem Knoten u bis $\text{captures}[\text{task}][u]$ iterativ mithilfe von `backtrack_Case0[u]` ins `vector<string> past3` gespeichert. Anschließend wird diese Liste reversiert, wodurch die Abfolge nicht mehr von u zur Anfangsknoten $\text{captures}[\text{task}][u]$ zeigt, sondern von der Anfangsknoten $\text{captures}[\text{task}][u]$ zur Knoten u zeigt. Anschließend werden die restlichen Bewegungsfolgen von der Knoten v zur Endknoten $\text{captures}[\text{task}][v]$ in die Liste aufgenommen, um es anschließend in `backtrack_Case1[captures[task][u]]` gespeichert wird.

2.2 Durchführung der Dial's Algorithmus für den Fall 2 und 3

Nachdem Fall 0 und Fall 1 komplett abgeschlossen wurden, werden für die Berechnung von Fall 2 und 3 folgende Variablen definiert:

- `vector<vector<int>> dist_case2(tasks + 2, N)`: Das gleiche wie `dist`.
- `vector<vector<int>> captures_case2(tasks + 2, N)`: Das gleiche wie `captures`.

Dadurch werden die Werte in der Dial's Algorithmus separat von Fall 0 und 1 gespeichert.

Für die Berechnung von Fall 2 und 3 werden die gleichen Verfahren wie die Berechnung von Fall 0 verwendet, wobei statt $z = 0$ für die Fälle 2 und 3 $z = 2$ gilt. Ein weiterer Unterschied dabei ist, dass nach dem Abschluss der Berechnung einer Arbeitsschrittgruppe für alle Werke u aus der nächsten Arbeitsschrittgruppe auf den Worst-Case Wert überprüft wird. Es gilt daher:

$$\text{DP}[u][2] = \max_{a(u)=\text{task}-1} (\text{DP}[u][2], \text{DP}[u][1])$$

Für Fall 3 wird die Formel aus der Lösungsidee wieder umgewandelt. Aus:

$$\text{DP}[u][3] = \text{dist}(u, v) + \text{DP}[v][3]$$

wird es mithilfe von:

$$v = \text{captures_case2}[\text{task}][v]$$

sowie:

$$\text{dist}(u, \text{captures_case2}[\text{task}][v]) = \text{dist_case2}[\text{task}][u] - \text{dist_case2}[\text{task}][\text{captures_case2}[\text{task}][v]]$$

zur:

$$\text{DP}[u][3] = \text{dist_case2}[\text{task}][u] - \text{dist_case2}[\text{task}][\text{captures_case2}[\text{task}][v]] + \text{DP}[\text{captures_case2}[\text{task}][v]][3]$$

³Ich verwende da String statt Int, da ich damit sofort das, was später ausgegeben werden sollte, speichern möchte und deshalb der Startknoten mit dem Index 0 zur S und der Endknoten mit dem Index $N - 1$ zur T umwandeln muss.

Diese Formel wird für die gleichen u und v Werte angewendet, wie die Werte aus Fall 2, die zu der minimalen Worst-Case Länge geführt hatten.

2.3 Endergebnis

Um zu bestimmen, ob die Lösung gültig ist, wird die Worst-Case Routenlänge am Startknoten S überprüft. Dabei wird geschaut, ob $DP[S][2] \leq \text{timelimit}$ wahr ist. Für den Fall, dass diese Aussage falsch ist, wird „unmöglich“ ausgegeben und das Programm beendet. Falls die Aussage wahr ist, werden die einzelnen Routenlängen gemäß der Lösungsidee ausgegeben.

Um die Weganweisungen zu generieren, wurden zwei Funktionen erschaffen:

- **getTrack**: Druckt unter der Verwendung von `backtrack_Case2` den empfohlenen Weg iterativ aus.
- **getBackupTrack**: Druckt unter der Verwendung von `backtrack_Case0`, Ersatzroute aus `backtrack_Case1` sowie dem Arbeitsschrittzustand `task` einen Ersatzweg iterativ aus.

3 Analyse

3.1 Laufzeit- und Speicheranalyse

Zum Thema Laufzeitanalyse werde ich ausschließlich den Teil betrachten, der für den größten Faktor verantwortlich ist, nämlich den DP-Teil kombiniert mit dem Dial's Algorithmus. Alle restlichen Teile, u.a. das Einlesen von Daten sowie das Ausgeben von Ergebnissen haben einen deutlich geringeren Einfluss auf die Laufzeit- sowie Speicherplatzkomplexität (als Beispiel würde ich die `getTrack` Funktion nehmen, die in einem Worst-Case Szenario eine Laufzeitkomplexität von $\mathcal{O}(N \cdot \text{tasks})$ besitzt, da in einem Worst-Case Szenario alle einzelnen Knoten aus allen Arbeitsschritten ausgegeben werden müssten. Es ist deutlich geringer als die Laufzeitkomplexität der DP-Teil).

Der Dial's Algorithmus besitzt eine Laufzeitkomplexität von $\mathcal{O}(V + E + V \cdot \text{maxWeight})^4$, da die Buckets in einem Worst-Case Fall bis $V \cdot \text{maxWeight}$ iterieren muss (durch den Einbau von `highDist` beträgt die tatsächliche Laufzeit oft geringer), die Knoten alle höchstens einmal besucht werden und alle Kanten höchstens zweimal verwendet werden (da alle Knoten nur einmal besucht werden und die Kanten nur zwei Knoten miteinander verbinden). Betrachtet man den Fall 0 sowie 1, so wird der Dial's Algorithmus `tasks`-mal durchgeführt, weshalb es sich zu $\mathcal{O}(\text{tasks} \cdot (V + E + V \cdot \text{maxWeight}))$ bildet, welches ebenfalls Fall 0 abdeckt, da die Bearbeitung von Fall 0 nur eine if-Abfrage zur Überprüfung der Arbeitsschrittgruppen mehr ist und daher eine Laufzeitkomplexität von $\mathcal{O}(1)$ in den Faktor hinzufügen würde.

Bei Fall 1 wird es aufgrund meiner Darstellung etwas auffälliger. Das liegt daran, dass beim Erfüllen der Fall 1 Bedingungen der Backtracking Wert berechnet wird. In der Theorie würde das dann heißen, dass bei jedem Erfüllen in einem Worst-Case Szenario bis zu V weitere Schritte berechnet werden müssen, jedoch ist es tatsächlich so, dass solch ein Worst-Case Szenario nahezu unmöglich ist, in der Graph umzusetzen. In einem Pfad-Graph würde der Fall 1 Bedienung für jede Knoten nur höchstens einmal für jede Knoten auslösen, aber kann dafür bis zu durchschnittlich $\frac{V}{2} + 1$ Schritte ausgeben, und in einem vollständigen Graph kann mithilfe von gezielter Problemmanipulation den Worst-Case Szenario öfters aufgerufen werden, jedoch würde in solchen Fällen nur $\mathcal{O}(1)$ Schritte für die Wiedergabe der Backtracking Ergebnisse verwendet werden und aufgrund der Definition von mögliche Gewichte: $\{1, \dots, 20\}$ kann es ebenfalls höchstens nur 19 Mal aufgerufen werden, weshalb der Effekt ebenfalls gering ist. Die Kombination der beiden Möglichkeiten kann die Effekte verstärken, jedoch wirken beide Faktoren konkurrierend zueinander (da die Erfüllung von $\frac{V}{2} + 1$ minimale Anzahl an Kanten (bzw. $V - 1$ Kanten) benötigt, während das gezielte mehrfache Aufrufen der Funktion eine maximale Anzahl an Kanten ($\sim V^2$ Kanten) benötigt), weshalb man ein Maximum finden kann, das jedoch nicht größer als die für die Berechnung der Fall 0 ist. Da der Fall 1 für jeden Knoten einmal berechnet wird, beträgt die endgültige Laufzeitkomplexität für Fall 1: $\mathcal{O}\left(V \cdot \left(\frac{V}{\sqrt{E}} \cdot C_2\right)\right)$, wobei C_2 für die durchschnittliche Anzahl der Auslösungen von Fall 1 pro Knoten steht. Der Faktor V muss durch den Faktor $\frac{E}{V}$ dividiert werden, da $\frac{V}{\sqrt{E}} = \frac{V}{\sqrt{V}} = V$ für $E = V$ und $\frac{V}{\sqrt{E}} = \frac{V}{\sqrt{V^2}} = 1$ für $E = V^2$ gelten muss. Dieser Faktor ist unabhängig von der Anzahl der Arbeitsschritte,

⁴vgl. <https://www.geeksforgeeks.org/dsa/dials-algorithm-optimized-dijkstra-for-small-range-weights/>. V steht hierbei für die Anzahl der Knoten im Graph G , E für die Anzahl der Kanten und `maxWeight` für das größte Gewicht aller Kanten im Graph G .

weshalb es sich zur $\mathcal{O}\left(\text{tasks} \cdot (V + E + V \cdot \text{maxWeight}) + V \cdot \left(\frac{V}{E} \cdot C_2\right)\right)$ bilden lässt.

Für die Berechnung von Fall 2 sowie 3 wird die gleiche Struktur wie die Berechnung von Fall 0 angenommen. Durch die Hinzufügung einer max-Funktion, welche für jeden Knoten einmal durchgeführt wird, summiert sich die Laufzeitkomplexität zu $\mathcal{O}(\text{tasks} \cdot (V + E + V \cdot \text{maxWeight}) + V)$, weshalb es sich zu folgender Gesamtlaufzeitkomplexität bilden lässt:

$$\mathcal{O}\left(2 \cdot \text{tasks} \cdot (V + E + V \cdot \text{maxWeight}) + V \cdot \left(\frac{V}{E} \cdot C_2\right) + V\right)$$

Da $V \ll V \cdot \text{maxWeight}$ gilt:

$$\mathcal{O}\left(\text{tasks} \cdot (E + V \cdot \text{maxWeight}) + V \cdot \left(\frac{V}{E} \cdot C_2\right)\right)$$

Für die Berechnung des Backtracking wird im Worst-Case bis zu $V \cdot \text{tasks}$ Schritte benötigt, was deutlich geringer als die Laufzeitkomplexität von Fall 0 sowie 2 und 3 ist, weshalb man es hierbei vernachlässigen kann. Der oben genannte Laufzeitkomplexität ist somit auch das endgültige Worst-Case Laufzeitkomplexität. Bei einer durchschnittlichen Laufzeitkomplexität kann man davon ausgehen, dass der Faktor $\frac{V}{E} \cdot C_2$ sehr gering ist, weshalb es im Durchschnitt sich zur folgenden Laufzeitkomplexität bilden lässt:

$$\mathcal{O}(\text{tasks} \cdot (E + V \cdot \text{maxWeight}))$$

Für die Berechnung mithilfe des Dial's Algorithmus werden mehrdimensionale Listen verwendet, die für jeden Arbeitsschritt sowie Knoten Werte speichert, um die Distanz von einem Knoten zu einem anderen Knoten zu bestimmen oder um die Wege per Backtracking wiederherzustellen. Aus diesem Grund besitzt es eine Speicherplatzkomplexität in der Höhe von $\mathcal{O}(\text{tasks} \cdot V)$. Für die Berechnung von Fall 1 werden die einzelnen Backtracking Werte ebenfalls direkt gespeichert. In einem Worst-Case Szenario (bzw. wenn der Graph G ein Pfad ist) werden in solchen Fällen $V \cdot \left(\frac{V}{2} + 1\right)$ Werte gespeichert, weshalb sich die Speicherplatzkomplexität in einem Worst-Case Szenario zu $\mathcal{O}(\text{tasks} \cdot V + V \cdot \frac{V}{2})$ bildet.⁵ In dem durchschnittlichen Szenario beträgt die Speicherplatzkomplexität $\mathcal{O}(\text{tasks} \cdot V)$.

3.2 Fragestellung Komplexitätseinstufung

Dieses Problem lässt sich eindeutig in der Kategorie P einstufen. Mithilfe des oben gezeigten Algorithmus ist es möglich, den besten Ergebnis innerhalb einer polynomialen Zeit zu berechnen. Die möglichen Änderungen um in die Einstufung NP zu kommen, werden in der Zusatzherausforderung erwähnt.

4 Werkzeuge

Es wurde kein ILP-Solver verwendet. Hinzu wurde bei der Lösungsideefindung, Umsetzung sowie Analyse keine KI verwendet.

5 Beispiele

Für die Beispiele wurden noch einige zusätzliche Beispiele erstellt, um Grenzszenarien darzustellen und um die Robustheit der Lösung zu beweisen. Alle Eingaben sowie Ausgaben sind im Ordner zu finden. Alle vorgegebenen Aufgaben stammen aus der Version vom 06.02.2026.

⁵Mit dem Gedanken, dass V höchstens 3932 betrug, kann man davon ausgehen, dass $V \cdot \frac{V}{2}$ weiterhin angemessen ist und für solche Sonderszenarien keine Maßnahmen nötig sind.

5.1 Vorgegebene Aufgaben

Aufgabe	Ausgabe	Kommentar
lieferung00.txt	MOEGLICH 15 9 S [3 1] [5 2] T 11 3 [2 1] 3 [5 2] T 15 5 T [4 2] T	Execution time: 914 μ s
lieferung01.txt	MOEGLICH 40 32 S [1 1] [4 2] [8 3] T 40 1 S [2 1] S 1 [4 2] [7 3] T 37 4 [5 2] [8 3] T 36 8 [6 3] T	Execution time: 484 μ s
lieferung02.txt	MOEGLICH 188 127 S [1 1] S [2 2] [4 3] 3 5 [6 4] 5 [7 5] 13 2 S 1 T 138 1 6 [5 1] [3 2] [4 3] 2 13 7 [10 4] [7 5] 13 2 S 1 T 157 2 13 7 5 [3 2] [4 3] 2 13 7 [10 4] [7 5] 13 2 S 1 T 188 4 3 5 6 12 [11 3] 9 8 [10 4] [7 5] 13 2 S 1 T 143 6 [14 4] 6 5 [7 5] 13 2 S 1 T 127 7 [13 5] 2 S 1 T	Execution time: 929 μ s
lieferung03.txt	UNMOEGLICH	Execution time: 4170 μ s
lieferung04.txt	MOEGLICH 264	Execution time: 4209 μ s
lieferung05.txt	MOEGLICH 930	Execution time: 72398 μ s
lieferung06.txt	UNMOEGLICH	Execution time: 300863 μ s

Aufgabe	Ausgabe	Kommentar
lieferung07.txt	MOEGlich 675	Execution time: 1044690µs
lieferung08.txt	MOEGlich 1333	Execution time: 1594679µs
lieferung09.txt	MOEGlich 11	Execution time: 318µs
lieferung10.txt	MOEGlich 1462	Execution time: 3468596µs

5.2 Eigene Aufgaben

Aufgabe	Ausgabe	Kommentar
1k_werk.txt	MOEGlich 36982	Aufgabe mit 1000 Arbeitsschritten. Execution time: 5657217µs Ausgabedatei ist ~11 MB groß, aus diesem Grund ist die Ausgabe nicht mit angeführt.
5k_werk.txt	MOEGlich 130775	Aufgabe mit 5000 Arbeitsschritten. Execution time: 103467607µs Ausgabedatei ist ~250 MB groß, aus diesem Grund ist die Aufgabe nicht mit angeführt.
10k_werk.txt	MOEGlich 24999	Aufgabe mit 10000 Arbeitsschritten. Ausgabedatei ist >1 GB groß, aus diesem Grund ist die Ausgabe nicht mit angeführt. Dieses Beispiel soll zeigen, dass das Programm auch für sehr hohe Eingabewerte funktioniert. (Die Ausführungsdauer habe ich leider nicht mehr, ich würde aber behaupten, dass es höchstens 45 Minuten gedauert hatte).
500_N.txt	MOEGlich 51 51 S [1 1] 272 698 697 [594 2] 43 93 92 861 383 384 385 T 51	Aufgabe mit 500 Werkstätten, die den gleichen Arbeitsschritt durchführen können. Execution time: 145206µs

Aufgabe	Ausgabe	Kommentar
	1 272 698 697 594 [43 1] 93 92 [861 2] 383 384 385 T 51 594 43 93 92 [861 2] 383 384 385 T	
2500_N.txt	MOEGLICH 52486	Aufgabe mit 2500 Werkstätten, die den gleichen Arbeitsschritt durchführen können. Execution time: 1398866µs
10k_N.txt	MOEGLICH 210828	Aufgabe mit 100000 Werkstätten, die den gleichen Arbeitsschritt durchführen können. Execution time: 11934050µs Das Programm funktioniert problemlos für sehr viele Knoten im Graph.
c_100.txt	MOEGLICH 5527	Graph mit Kantenlängen von bis zu 100. Execution time: 602808µs Durch die Erhöhung der Kantengewichte benötigen die Berechnungen etwas mehr Zeit, jedoch werden die trotzdem korrekt berechnet. Bei sehr hohen Kantengewichten sollte lieber Dijkstra verwendet werden.
unreachable.txt	UNMOEGLICH	Es gibt keinen Weg zwischen Startknoten und Zielknoten. Execution time: 15800µs Das Programm stürzt nicht ab.
vollstaendigerGraph.txt	MOEGLICH 73	Vollständiger Graph. Execution time: 1266066µs Trotz hoher Kantenzahl kann das Programm problemlos eine Lösung ausgeben.
wis1.txt	UNMOEGLICH	Für bestimmte Arbeitsschritte gibt es nur 1 Werk. Execution time: 154335µs Das Programm stürzt ebenfalls nicht ab.

6 Quellcode

Aus Platzgründen wurden unnötige Sachen stark verkürzt (gekennzeichnet durch [...]) oder weggelassen. Die wichtigsten Strukturen sind weiterhin vorhanden. Für den vollständigen Quellcode, sollte die im Abgabe-Ordner findbare Quelltext Datei angeschaut werden.

```
void solve() {
    int timelimit;
    cin >> timelimit;
    int tasks;
    cin >> tasks;
    [...] // Redundanz
    int w;
    cin >> w;
    int N = w + 2;
    vector<vector<int>> ids(tasks + 2);
    vector<int> rev_ids(N);
    ids[0].push_back(0); // Startknote
    rev_ids[0] = 0;
    ids[tasks + 1].push_back(N - 1); // Endknote
    rev_ids[N - 1] = tasks + 1;
    [...] // Einlesen der Daten für ids sowie rev_ids (Einlesen der Arbeitsschritte)
    int M;
    cin >> M;
    vector<vector<pair<int, int>>> adj(N);
    int maxWeight = 0;
    [...] // Einlesen der Kanten für den Graph G
    vector<vector<int>> DP(N, vector<int>(4, 1e9));
    /*
        DP States:
        DP[v][0]: Das Betrag der kürzeste Weg von Knote v zur Endknote unter Beachtung von
        Werkstätten, die noch besucht werden müssen.
        DP[v][1]: Das Betrag der kürzeste Weg von Knote v zur Endknote über einen
        Alternativwerkstatt, unter Beachtung von Werkstätten die noch besucht werden müssen.
        Der Weg wird wie gefolgt definiert: DP[v][1] = min(dist(v, w) + DP[w][0]) wobei w
        ein Werkstatt darstellt, welches die gleiche Arbeitsprozess wie v durchführen kann.
        DP[v][2]: Das Betrag der worst Case Weg von Knote v zur Endknote über alle kürzeste
        Ersatzwege.
        Der Weg wird wie gefolgt definiert: DP[v][2] = max(min(dist(v, w) + DP[w][2]),
        DP[v][1]) wobei w für ein Werkstatt darstellt, welches die darauf folgende Arbeitsprozess
        durchführen kann
        DP[v][3]: Der tatsächliche Betrag der länge des Weges, welches durch DP[v][2] bestimmt
        wurde, unter der Beachtung von Werkstätten, die noch besucht werden müssen.
    */

    [...] // Alle DP-Werte der Endknote werden auf 0 gesetzt.

    // dial's algorithm (Code abgeändert aus https://www.geeksforgeeks.org/dsa/dials-
    algorithm-optimized-dijkstra-for-small-range-weights/)

    vector<vector<int>> dist(tasks + 2, vector<int>(N, 1e9));
    int maxDist = (N - 1) * maxWeight * 3;
    vector<vector<int>> buckets(maxDist + 1);

    // Verweist alle Knoten auf ihre Anfangsknoten
    vector<vector<int>> captures(tasks + 2, vector<int>(N, -1));

    // kürzester Weg von aktuelle Knote zur Endknote (unter Beachtung von Werkstätten, die
    noch besucht werden müssen) (Fall 0)
    vector<vector<int>> backtrack_Case0(tasks + 2, vector<int>(N, -1));
    // Für jedes Werkstatt ein alternativwerkstatt (Fall 1)
    vector<vector<string>> backtrack_Case1(N);
```

```

// Der beste Weg für den fall, dass keine Werke streiken. (Fall 2, 3)
vector<int> recommendedPaths(N, -1);

for (int task = tasks + 1; task > 0; task--) {
    // Vorbereitung Beginn
    int bias = 1e9;
    for (int id : ids[task]) {
        bias = min(bias, DP[id][0]);
    }
    int highDist = 0;
    for (int id : ids[task]) {
        dist[task][id] = DP[id][0];
        buckets[DP[id][0] - bias].push_back(id);
        highDist = max(highDist, DP[id][0] - bias);
        captures[task][id] = id;
    }
    // Vorbereitung Ende
    for (int d = 0; d <= maxDist && d <= highDist; d++) {
        while (!buckets[d].empty()) {
            int u = buckets[d].back();
            buckets[d].pop_back();

            if (d > dist[task][u] - bias) continue;
            for (auto& [v, weight] : adj[u]) {
                int newDist = dist[task][u] + weight;

                // Case 1 Bearbeitung
                if (captures[task][v] != -1 && captures[task][v] != captures[task][u]) {
                    // Überprüfung Knoten v als Alternativknoten für Knoten u
                    if (dist[task][u] + weight - dist[task][captures[task][u]] +
dist[task][v] < DP[captures[task][u]][1]) {
                        DP[captures[task][u]][1] = dist[task][u] + weight - dist[task]
[captures[task][u]] + dist[task][v];
                        // Backtrack Berechnung von u zur v.
                        [...] // Berechnung der Backtracking
                    }
                    // Überprüfung Knoten u als Alternativknoten für Knoten v
                    if (dist[task][v] + weight - dist[task][captures[task][v]] +
dist[task][u] < DP[captures[task][v]][1]) {
                        DP[captures[task][v]][1] = dist[task][v] + weight - dist[task]
[captures[task][v]] + dist[task][u];
                        // Backtrack Berechnung von v zur u.
                        [...] // Berechnung der Backtracking
                    }
                }
            }

            // Überprüfung neuer kürzester Knoten
            if (newDist < dist[task][v]) {
                dist[task][v] = newDist;
                backtrack_Case0[task][v] = u;
                buckets[newDist - bias].push_back(v);
                highDist = max(highDist, newDist);
                captures[task][v] = captures[task][u];
                // Case 0 Bearbeitung
                if (rev_ids[v] == task - 1) {
                    if (newDist < DP[v][0]) {
                        DP[v][0] = newDist;
                    }
                }
            }
        }
    }
}

```

```

    }
}

// Separate Trennung der Variablen für den zweiten Dial's Algorithmus Aktivierung
vector<vector<int>> dist_case2(tasks + 2, vector<int>(N, 1e9));
vector<vector<int>> captures_case2(tasks + 2, vector<int>(N, -1));
// Empfohlener Weg von Startknoten S zur Endknoten T unter der Beachtung von
Arbeitsschritten. (Fall 2 und 3)
vector<vector<int>> backtrack_Case2(tasks + 2, vector<int>(N, -1));

for (int task = tasks + 1; task > 0; task--) {
    // Vorbereitung Beginn
    int bias = 1e9;
    for(int id : ids[task]) {
        bias = min(bias, DP[id][2]);
    }

    int highDist = 0;
    for (int id : ids[task]) {
        dist_case2[task][id] = DP[id][2];
        buckets[DP[id][2] - bias].push_back(id);
        highDist = max(highDist, DP[id][2] - bias);
        captures_case2[task][id] = id;
    }
    // Vorbereitung Ende
    for (int d = 0; d <= maxDist && d <= highDist; d++) {
        while (!buckets[d].empty()) {
            int u = buckets[d].back();
            buckets[d].pop_back();
            if (d > dist_case2[task][u] - bias) continue;
            for (auto& [v, weight] : adj[u]) {
                int newDist = dist_case2[task][u] + weight;

                if (newDist < dist_case2[task][v]) {
                    dist_case2[task][v] = newDist;
                    backtrack_Case2[task][v] = u;
                    buckets[newDist - bias].push_back(v);
                    highDist = max(highDist, newDist);
                    captures_case2[task][v] = captures_case2[task][u];

                    // Überprüfung: Fall 2 und 3
                    if (rev_ids[v] == task - 1) {
                        if (newDist < DP[v][2]) {
                            DP[v][2] = newDist;
                            DP[v][3] = dist_case2[task][v] - dist_case2[task]
[captures_case2[task][u]] + DP[captures_case2[task][u]][3];
                            recommendedPaths[v] = captures_case2[task][u];
                        }
                    }
                }
            }
        }
    }
}

// Fall 2: Ist der aktuelle Alternativweg länger oder die Wegen zu alle bisherigen
Alternativwegen?
if (task - 1 != 0) {
    for (auto id : ids[task - 1]) {
        if (DP[id][2] < DP[id][1]) {
            DP[id][2] = DP[id][1];
        }
    }
}

```

```

    }
}

// Falls das Betrag der worst case Streckenlänge das Zeitlimit überschreiten sollte.
if (DP[0][2] > min(timelimit, (int) 1e9 - 1)) {
    fail();
    return;
}
cout << "MOEGlich" << "\n";
// Worst-Case Dauer
cout << DP[0][2] << "\n";
cout << "\n";
// empfohlener Weg
cout << DP[0][3] << "\n";
getTrack(backtrack_Case2);
// Ausgabe aller Alternativwegen
int pos = recommendedPaths[0];
int task = 1;
while (pos != N - 1) {
    cout << DP[0][3] - DP[pos][3] + DP[pos][1] << "\n";
    getBackupTrack(backtrack_Case0, backtrack_Case1[pos], task);
    task++;
    pos = recommendedPaths[pos];
}
cout << flush;
}

```

7 Zusatzherausforderung: Bitmask Lieferprobleme

Aufgrund des technischen Fortschritts müssen alle Arbeitsschritte nicht mehr in der Reihenfolge durchgeführt werden. Aus diesem Grund möchte das Management eine neue Berechnung haben, die alle Arbeitsschritte beinhaltet, jedoch müssen die Arbeitsschritte nicht mehr in der Reihenfolge durchgeführt werden.

Die Schwierigkeit besteht nun, dass durch diese Veränderung deutlich mehr mögliche Zustände von Arbeitsschritten gibt. Statt zu sagen, wie viele Arbeitsschritte schon bereits durchgeführt wurden - um dadurch eindeutig sagen zu können, welcher Schritt der nächste ist (vgl. Kapitel 1.2 Lösungsidee), muss man jetzt sagen, welche Arbeitsschritte schon bereits durchgeführt wurden und welche Arbeitsschritte noch durchzuführen sind. Dabei gibt es nicht mehr nur einen eindeutigen Arbeitsschritt, sondern mehrere mögliche Arbeitsschritte, die man durchführen könnte.

Im Folgenden werden die Lösungsidee, Umsetzung, Analyse sowie Beispiele separat von der Hauptlösung ausformuliert.

8 Zusatzherausforderung: Lösungsidee

8.1 Graphenbildung

Wir bilden den ungerichteten gewichteten Graphen $G = (V, E)$ mit den Gewichten $w : E \rightarrow \{1, 2, \dots, 20\}$. Zusätzlich bilden wir die Menge $A = \{0, 1, \dots, k, k+1\}$ mit $a : V \rightarrow A$ welches die auszuführenden Arbeitsschritte darstellt, wobei k für die Gesamtzahl der Arbeitsschritte steht und zugleich folgende Eigenschaften gelten:

- $a^{-1}(0) = \{S\}$ (Startknoten)⁶
- $a^{-1}(k+1) = \{T\}$ (Zielknoten)
- $a^{-1}(i) \subset V \forall i \in \{1, 2, \dots, k\}$ die Menge der Werkstätten ist, die die Aufgabe i erfüllen können.

Mithilfe des Dial-Algorithmus (ein kürzester Weg Algorithmus)⁷ bilden wir die Funktion $\text{dist}(u, v)$, $u \in V$; $v \in V$, die für die Knoten u und v die kürzeste Länge zurückgibt. Da hierbei die Länge zwischen allen Knoten benötigt werden, wird aus diesem Grund der Dial's Algorithmus mithilfe des Graphen G sowie

⁶vgl. [https://de.wikipedia.org/wiki/Urbild_\(Mathematik\)](https://de.wikipedia.org/wiki/Urbild_(Mathematik))

⁷vgl. <https://www.geeksforgeeks.org/dsa/dials-algorithm-optimized-dijkstra-for-small-range-weights/>

die Kantengewichte w so oft ausgeführt, wobei jeder Knoten einmal als Startpunkt ausgewählt wird, um am Ende alle Längen untereinander zu berechnen zu können.

8.2 Dynamische Programmierung

Um die Lösung zu bestimmen, führen wir hier wieder die kürzeste Wegberechnung aus dem Endknoten T aus. Das liegt hier wieder daran, dass wir dadurch für jede Knoten die Frage stellen können: „Wie weit bin ich aus der aktuellen Knoten u von der Endknoten T entfernt?“ und daraus die Frage ableiten können: „Wie weit bin ich aus der aktuellen Knoten u von der Endknoten T entfernt, wenn ich über einen Alternativ-Weg gehen würde?“. Solche Fragestellung wäre nicht möglich, wenn man es von der Startknoten S berechnen würde, da dort die Frage heißen würde: „Wie weit habe ich mich von der Startknoten S bewegt, um bei der Knoten u anzukommen.“, die man nicht effektiv zu der genannten Alternative-Weg Frage ableiten kann. Wir erstellen daher den $DP : \text{mask} \times u \times z \rightarrow \mathbb{N}_0$, wobei $\text{mask} \subseteq A \setminus \{0, k+1\}$, $u \in V$ und $z \in \{0, 1, 2, 3\}$ gilt.

- Der mask beschreibt die bereits durchgeführten Arbeitsschritte. Dabei wird der Wert von mask aus Sicht der Informatik als Binär dargestellt (wobei jedes Bit für einen bestimmten Arbeitsschritt steht), während ich es mathematisch als eine Menge darstellen lasse. Für das Besuchen der Knoten u gilt:

$$\text{newMask} = \text{mask} \cup \{a(u)\}$$

Für die Initialisierung gilt:

$$DP(\emptyset, T, 0) = 0$$

$$DP(\emptyset, T, 1) = 0$$

$$DP(\emptyset, T, 2) = 0$$

$$DP(\emptyset, T, 3) = 0$$

$$\text{sonst: } DP(\text{mask}, u, z) = \infty$$

Mithilfe von z werden die einzelnen Fallunterscheidungen bearbeitet. Dabei werden die Fälle für $z = 0$ aufsteigend bis 3 mithilfe von Kombinatorik bearbeitet.

8.2.1 Der direkte Weg (Fall 0)

Unter der Beachtung von noch auszuführenden Arbeitsschritten berechnet $DP[\text{mask}][u][0]$ die kürzeste Länge von u zum Endknoten T . Für die Berechnung gilt:

$$DP(\text{mask} \cup \{a(u)\}, u, 0) = \min_{\substack{i \leq k \\ i=1, u \in a^{-1}(i), i \notin \text{mask}, a(v) \in \text{mask}}} (\text{dist}(u, v) + DP(\text{mask}, v, 0))$$

8.2.2 Der kürzeste Weg über ein Alternativwerk (Fall 1)

Unter der Beachtung von noch auszuführenden Arbeitsschritten berechnet $DP(\text{mask}, u, 1)$ einen kurzen Weg von u zum Endknoten T über einen Alternativknoten v . Für die Berechnung gilt:

$$DP(\text{mask}, u, 1) = \min_{a(u)=a(v), u \neq v} (\text{dist}(u, v) + DP(\text{mask}, v, 0))$$

Hinweis: Diese Berechnung wird nicht immer den absoluten kürzesten Weg garantieren, da ein kürzerer Weg über andere Arbeitsschritte (statt der Arbeitsschritte aus dem streikenden Werk) möglich sein könnten. Jedoch würde eine solche Implementierung die Laufzeit- und Speicherplatzkomplexität um den Faktor N erhöhen, da alle Berechnungen unter Deaktivierung eines Knotens durchgeführt werden müssten. Die Lösung sollte aber in den meisten Fällen weiterhin gut genug sein, um brauchbare Ergebnisse zu bekommen.

8.2.3 Der optimale Weg (Fall 2 und 3)

$DP(\text{mask}, u, 2)$ bestimmt den empfohlenen Weg, der genommen werden soll, um die Worst-Case Zeit zu minimieren. $DP(\text{mask}, u, 3)$ zeichnet dabei die tatsächliche Länge der berechneten Strecke aus Fall 2 auf. Für die Berechnung gilt:

$$\begin{aligned}
& \text{DP}(\text{mask} \cup \{a(u)\}, u, 2) = \\
& = \max \left(\min_{\substack{i \leq k \\ i=1, u \in a^{-1}(i), i \notin \text{mask}, a(v) \notin \text{mask}}} \{ \text{dist}(u, v) + \text{DP}(\text{mask}, v, 2) \}, \text{DP}(\text{mask} \cup \{a(u)\}, u, 1) \right) \\
& \text{DP}(\text{mask} \cup \{a(u)\}, u, 3) = \text{dist}(u, v) + \text{DP}(\text{mask}, v, 3)
\end{aligned}$$

Dabei ist wichtig zu erwähnen, dass die beiden v für Fall 2 und 3 die gleichen Werte besitzen müssen.

8.3 Backtracking

Um die Wege auszugeben, werden während der DP-Verfahren für alle Knoten die Knoten, die zu dem besten Ergebnis geführt hatten, gespeichert, um die einzelnen Knoten später iterativ wieder auszugeben. Es gelten daher für Fall 0: `defaultPaths[mask][u]` (kürzeste Wegbeschreibung vom Knoten u zum Endknoten T), für Fall 1: `backups[mask][u]` (kürzeste Wegbeschreibung für eine nahe Alternativfabrik, die die gleichen Arbeitsschritte $a(u)$ durchführen kann) und für Fall 2: `recommendedPaths[mask][u]` (kürzeste Wegbeschreibung für den minimalen Worst-Case Zeit). Hinzu wird die Funktion `Backtrack(u, v)` definiert, die während des Dial's Algorithmus Verfahrens den kürzesten Weg von v zu u speichert. D.h. durch die Verwendung von `Backtrack(u, v)` erhalten wir einen Knoten, der sich dem Knoten u annähert. Es gilt: `Backtrack(u, u) = u`

Dadurch erhalten wir während der Weg-Ausgabe aus Sicht eines Knotens u den Ort für den nächsten Arbeitsschritt durch `defaultPaths[mask][u]`, `backups[mask][u]` und `recommendedPaths[mask][u]` und den tatsächlichen Weg mithilfe von `Backtrack(u, v)`, wobei v eine Ausgabe von einem der drei obigen `Backtrack`-Speicherobjekte ist.

8.4 Längenberechnungen

Um die Länge auszugeben, werden die einzelnen Einträge aus dem DP-Verfahren verwendet.

- Für die absolute Worst-Case Länge entspricht der Wert:

$$\text{DP}(A \setminus \{0, k+1\}, S, 2)$$

- Für die Länge aus dem Startknoten S (mit der empfohlenen Wegbeschreibung) entspricht der Wert:

$$\text{DP}(A \setminus \{0, k+1\}, S, 3)$$

- Für die Gesamtlänge bei einem Streikfall bei der Knoten u entspricht der Wert:

$$\text{DP}(A \setminus \{0, k+1\}, S, 3) - \text{DP}(\text{mask}, u, 3) + \text{DP}(\text{mask}, u, 1)$$

9 Zusatzherausforderung: Umsetzung

Die Lösung wird hier ebenfalls in C++ umgesetzt. Im Folgenden werden für einige Variablen deren Bezeichnung mit deren Bedeutung aufgelistet. Einige weitere Variablen, die nicht in der Liste enthalten sind, werden später ausführlicher beschrieben:

- `timelimit`: Maximale erlaubte Worst-Case Zeit gemäß Aufgabenstellung
- `tasks`: Anzahl der Arbeitsschritte.
- `N`: Gesamtzahl der Knoten im Graph G . Wird mithilfe der Formel $w + 2$ berechnet, wobei w in diesem Fall für die Anzahl der Fabriken steht.

Dabei wurde die Funktion `void solve()` erstellt, die die einzelnen Probleme lösen kann.

- Die Funktionen für die Arbeitsschritte $a(u)$ werden mithilfe von `vector<int> rev_ids` mit der Länge N erstellt und die einzelnen Einträge reingeladen.
- Das Urbild $a^{-1}(u)$ wird mithilfe von `vector<vector<int>> ids` erstellt, mit der Länge $k + 2$, wobei k für die Anzahl der Gesamtarbeitsschritte steht.
- Der Graph G wird ebenfalls geladen, die Anzahl der Knoten unter N gespeichert und die einzelnen Kanten werden in der Adjazenzliste `vector<vector<pair<int, int>>> adj` gespeichert, wobei für jeden Knoten u eine Liste zu seinen Nachbarn v mit deren Kostenfunktion w (Länge der benachbarten Verbindung zwischen u und v) gespeichert wird.
- Der Startknoten S wird als Knoten 0 definiert und die Endknoten als Knoten $N - 1$.

Um alle Längen zwischen zwei beliebigen Knoten u und v zu berechnen, wird der Dial's Algorithmus auf alle N Knoten angewendet. Dadurch kann die Distanzfunktion $\text{dist}(u, v)$ in C++ durch die Adjazenzmatrix `vector<vector<int>> dist` erstellt werden. Der Code wurde dabei von <https://www.geeksforgeeks.org/dsa/dials-algorithm-optimized-dijkstra-for-small-range-weights/> übernommen

- Alle Ergebnisse werden in der Vektor `dist[u][v]` gespeichert, wobei u der Startknoten ist. Dadurch wird der Betrag des kürzesten Wegs zwischen zwei beliebigen Knoten u und v gespeichert und erlaubt später einen schnellen Zugriff.
- Um den Dial's Algorithmus zu optimieren, wurden ein paar Änderungen vorgenommen. `int highDist` speichert die Distanz des weitesten Knotens, um sicherstellen zu können, dass nach der Berechnung nicht mehr auf weitere Buckets zugegriffen werden, die garantiert leer sind.
- Hinzu werden alle Vorgänger in `backtrack[u][v]` gespeichert, wobei u der Startknoten ist. Dadurch ist eine spätere Wegbeschreibung mit dem kürzesten Weg von irgendeinem Knoten v zu Knoten u möglich.

Um den Bitmask darzustellen, werden $2^{\text{tasks} + 1}$ mögliche Zustände erstellt. Dies erlaubt uns, alle möglichen Zustände aus $\text{mask} \subset A \setminus \{0, k + 1\}$ ⁸ darzustellen. Das Besuchen eines Knotens u von:

$$\text{newMask} = \text{mask} \cup \{a(u)\}$$

entspricht daher:

$$\text{newMask} = \text{mask} \mid (1 \ll (\text{rev_ids}[u] - 1))$$

- Die leere Menge $\emptyset$ entspricht dem Wert 0.

Um alle Wegbeschreibungen zu speichern, werden alle Variablen, die für Backtracking verwendet werden, wie folgt gespeichert:

- `vector<vector<int>> backups[1 << (tasks + 1)][N]`
- `vector<vector<int>> defaultPaths[1 << (tasks + 1)][N]`,
- `vector<vector<int>> recommendedPaths[1 << (tasks + 1)][N]`
- `vector<vector<vector<int>>> DP[1 << (tasks + 1)][N][4]`
- Die erste Dimension stellt bei allen die möglichen Zustände der Arbeitsschritte dar und die zweite Dimension stellt die Knoten selbst dar.

Der Fall $\text{mask} = 0$ wird bei mir separat verarbeitet, da ich aufgrund meiner Bitverarbeitungen den Fall nicht bearbeiten kann und dem Endknoten T keinen Arbeitsschritt zugewiesen habe. Die Wege für $\text{mask} = 0$ berechnen sich daher aus:

- $\text{DP}[1 \ll \text{rev_ids}[i]][0]_{i=1}^{i \leq N-1} = \text{dist}[i][N - 1]$
- $\text{defaultPaths}[1 \ll \text{rev_ids}[i]][i]_{i=1}^{i \leq N-1} = N - 1$

Die mask -Zustände werden anschließend von 0 aufsteigend bis $(1 \ll \text{tasks})$ gezählt und dabei wird der Fall 0 gefolgt von dem Fall 1 bearbeitet. Anschließend wird mithilfe der Variable „task“ die einzelnen Arbeitsschritte von 0 bis tasks hochgezählt, um die noch nicht durchgeführten Arbeitsschritte zu notieren. Bereits durchgeführte Arbeitsschritte werden mithilfe der Regel $\text{mask} \& (1 \ll \text{task}) == 0$ herausgefiltert. Mithilfe von Binary Tricks werden die bereits durchgeführten Arbeitsschritte wie folgt extrahiert:

```
int c = mask;
while (c) {
    int Fabrik = LOG2(c & -c);
    c -= c & -c;
    ...
}
```

Dadurch können wir die Knoten u auswählen, die als ein gültiger Ausgangspunkt gesehen werden kann, wodurch alle weiteren Arbeitsschritte für den Fall 0 mithilfe von Kombinatorik berechnet werden können, wobei der günstigste Fabrik im `defaultPaths` gespeichert wird.

Sobald für einen Mask-Zustand alle die besten Fall 0 Ergebnisse berechnet wurden, wird der Fall 1

⁸Vgl. Lösungsidee

mithilfe einer aufsteigend zählenden Task-Zustände für den gleichen Mask-Zustand komplett berechnet, wobei diesmal nur die Task-Zustände erlaubt sind, für die $\text{mask} \&(1 \ll \text{task}) \neq 0$ gilt.

Mithilfe von Kombinatorik wird wieder die günstigste Alternativ-Fabrik berechnet und für alle Fabriken eine günstige Alternativ Fabrik im `backups` zugewiesen.

Sobald die Probleme für Fall 0 und Fall 1 bearbeitet sind, werden Fall 2 und Fall 3 bearbeitet. Die Fälle werden ähnlich wie Fall 0 bearbeitet, mit dem Unterschied, dass bei den Bitmask-Zuständen für alle gültigen Knoten zusätzlich noch überprüft wird, ob der bisherige Worst-Case Weg (Fall 2 Weg) kürzer ist, als die aktuelle Alternativ-Fabrik Weg (Fall 1 Weg).

```
DP[mask][u][2] = max(DP[mask][u][2], DP[mask][u][1]);
```

Dadurch können die Worst-Case Weg korrekt berechnet werden und minimiert werden. Die Nachfolgerknoten, die einen minimalen Worst-Case Weg ermöglichen, werden unter `recommendedPaths` gespeichert.

Nach der Berechnung werden die Fälle 2 und 3 für den Startknoten S aufgrund von Limitierungen mit der Bitmask-Umsetzung separat berechnet und dabei alle gültigen Kombinationen beachtet.

Mithilfe der in der Lösungsidee genannten Formeln werden am Ende die Routenlänge ausgegeben, wobei der Worst-Case Routenlänge mit dem `timelimit` verglichen wird, um festzustellen, ob das Problem möglich ist. Die Wegbeschreibungen selbst werden mithilfe der Funktion `getTrack` (für den empfohlenen Weg) sowie `getBackupTrack` (Wegbeschreibung bei Ausfällen) ausgegeben. `getTrack` verwendet dabei die `recommendedPaths` (Fall 2), um von dem Startknoten S die Werke zu bestimmen und `backtrack`, um die Wegbeschreibungen von zwei beliebigen Knoten zu erhalten. `getBackupTrack` verwendet die verwendeten `defaultPaths` (Fall 0) sowie die möglichen Ausfall-Standorte (`recommendedPaths`, Fall 2) mit dem Unterschied, dass mithilfe von `backups` (Fall 1) für den ersten Zielknoten ein Alternativwerk als Ziel gesetzt wird, bevor es wie `getTrack` eine Wegbeschreibung ausgibt.

10 Zusatzherausforderung: Analyse

10.1 Laufzeit und Speicherplatzkomplexität

Die Lösung kann auf die zwei größten Faktoren aufgeteilt werden:

1. Dial's Algorithmus
2. DP mit Bitmask

Die restlichen Faktoren sind deutlich geringer, weshalb sie vernachlässigt werden.

10.1.1 Dial's Algorithmus

Der Dial's Algorithmus berechnet hierbei aus einem Startknoten $S \in V$ den kürzesten Weg für alle $N = |V|$ Knoten des Graphen $G(V, E)$ mit der Gewichtsfunktion $w : E \rightarrow \{1, \dots, 20\}$. Der Dial's Algorithmus hat eine Laufzeitkomplexität von $\mathcal{O}(N \cdot w)$.⁹ Da wir den Algorithmus N mal durchführen, hätten wir somit eine Laufzeitkomplexität in der Höhe von $\mathcal{O}(N \cdot (N \cdot w))$. Im Bereich der Speicherplatzkomplexität hat der Dial's Algorithmus eine Speicherplatzkomplexität von $\mathcal{O}(N \cdot w)$, da jedoch für alle N Knoten die kürzesten Distanzen zu den restlichen N Knoten gespeichert werden, hat man somit insgesamt eine Speicherplatzkomplexität in der Höhe von $\mathcal{O}(N \cdot w + N^2)$

10.1.2 DP mit Bitmask

Hinweis: Die Fälle 0, 1, 2 und 3 besitzen alle eine ähnliche äußere Struktur zueinander, weshalb ich die Laufzeitkomplexität für Fall 0 begründe. Die Ergebnisse aus Fall 0 sollten aufgrund der Ähnlichkeit ebenfalls für Fall 1, 2 sowie 3 gelten bzw. die Komplexitäten sind für die Fälle sogar kleiner (z.B. der Fall 1). Der DP Teil berechnet für jede mögliche Bitmask Zustand den kürzesten Weg und das für jeden möglichen Knoten zu den restlichen möglichen Knoten. Aus diesem Grund, hat es in einem worst-case Szenario eine Laufzeitkomplexität von höchstens $\mathcal{O}(2^{\text{tasks}} \cdot N^2)$, wobei der tatsächliche Laufzeitkomplexität deutlich niedriger liegt. Bei genauerer Analyse fällt es einem auf, dass es extrem von der Anzahl der Arbeitsschritte abhängig ist (wir haben einmal einen Innerloop mit `task` und einen von Bitmask

⁹vgl. <https://www.geeksforgeeks.org/dsa/dials-algorithm-in-python/>

abhängigen Bereits-Besucht Status, die ebenfalls von der Anzahl der Arbeitsschritte abhängig ist). Die tatsächliche Kombinatorik Teil kombiniert alle Kombinationen mit zwei verschiedenen Gruppen, wobei eine Gruppe alle die gleichen Arbeitsschritte machen können. Aus diesem Grund beträgt die Laufzeitkomplexität von ungefähr $\mathcal{O}(2^{\text{tasks}} \cdot \text{tasks} \cdot L^2)$, wobei L für die durchschnittliche Gruppengröße von Knoten darstellt, die die gleichen Arbeitsschritte durchführen können. (Alternativ mathematisch: $L = \frac{N-2}{\text{tasks}})^{10}$

Im Bereich der Speicherplatzkomplexität beträgt es $\mathcal{O}(2^{\text{tasks}} \cdot N)$, da wir für alle möglichen Bitmask Zustände für jede Knoten Werte einspeichern müssen.

10.1.3 Ergebnis

Die Laufzeitkomplexität beträgt $\mathcal{O}(N \cdot N \cdot w + 2^{\text{tasks}} \cdot \text{tasks} \cdot L^2)$.

Die Speicherplatzkomplexität beträgt $\mathcal{O}(N \cdot w + N^2 + 2^{\text{tasks}} \cdot N)$.

10.2 Problemeinstufung

Dieses Problem lässt sich eindeutig in der NP-Vollständig einstufen. Mithilfe von Reduktion ist es möglich, dieses Problem auf den „Travelling Salesman Problem“ (kurz TSP)¹¹ zu reduzieren, welches per Definition NP-Schwer ist. Die Definition des Problems lautet: „Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?“. ¹² Natürlich muss man sich dabei des Bewusstseins sein, dass man nicht zu seinem Startknoten S zurückkehrt (sondern bei dem Endknoten T aufhört) und gleichzeitig nicht alle Werke besuchen wird, weshalb sich diese Definition ohne Begründung nicht vollständig anwenden lässt.

Jedoch können wir diese Definition trotzdem anwenden, da das TSP-Problem auf das Hamilton-Kreis Problem basiert,¹³ welches ebenfalls NP-Vollständig ist.¹⁴ Durch die Trennung von Startknoten S und Endknoten T kann dieses Hamilton-Kreis Problem auf das Hamilton Pfad Problem umgewandelt werden, was die Komplexität des Problems nicht verändert.¹⁵

Nun muss nur noch argumentiert werden, inwiefern man alle Städte im Problem besuchen wird. In der erweiterten Problemstellung wird erwähnt, dass jeder Arbeitsschritt unabhängig von der Reihenfolge durchgeführt werden muss. Würde man daher die einzelnen Arbeitsschritte als eine mehrdimensionale Knoten - die aus Knotengruppen¹⁶ von Werken bestehen - sehen, wobei die Kanten von den einzelnen Knoten vererbt werden, so hätten die einzelnen Arbeitsschritte-Knoten die gleichen Eigenschaften wie die Städte aus dem TSP-Problem.

Hinzu wäre dadurch möglich (unter der Annahme, dass der Graph G ein zusammenhängender Graph ist), einen vollständigen Graphen $G_2 = (V_2, E_2)$ zu bilden, wobei V_2 die Arbeitsschritte-Gruppen selbst sind und E_2 (abhängig von den Knoten) die direkte, kürzeste Distanz zwischen zwei Arbeitsschritten darstellt, wodurch die Eigenschaften eines TSP-Problems nachgestellt werden.

Somit sollte es deutlich sein, dass das Problem auf das TSP reduziert werden kann und das Finden einer Lösung in der Kategorie NP zugeordnet werden kann.

Jedoch bleibt nur noch eine offene Frage zu klären: Das TSP-Problem ist NP-Schwer und das Lieferkettenproblem ist NP-Vollständig.

Die Überprüfung beim TSP, ob ein gegebener Weg W der absolut kürzeste Weg ist, ist in einer polynomiellen Zeit laut dem aktuellen Standpunkt nicht möglich, weshalb es NP-Schwer ist. Jedoch wird in der Lieferkette Problem nach einem Weg W gesucht, der einen bestimmten Längenbetrag nicht überschreiten

¹⁰ $N - 2$, da der Startknoten S sowie der Endknoten T nicht dazu zählen.

¹¹vgl. https://en.wikipedia.org/wiki/Travelling_salesman_problem

¹²https://en.wikipedia.org/wiki/Travelling_salesman_problem, geöffnet am 31.03.2026

¹³ „An equivalent formulation in terms of graph theory is: Given a complete weighted graph [...], find a Hamiltonian cycle with the least weight.“, vgl. https://en.wikipedia.org/wiki/Travelling_salesman_problem

¹⁴Vgl. https://en.wikipedia.org/wiki/Hamiltonian_path

¹⁵ „The requirement of returning to the starting city does not change the computational complexity of the problem; see Hamiltonian path problem.“, https://en.wikipedia.org/wiki/Travelling_salesman_problem, geöffnet am 31.03.2026

¹⁶Die Knoten werden als eine Gruppe gesehen, da für die Berechnung von Fall 1 sowie Fall 2 zwei Knoten aus der gleichen Arbeitsschritt-Gruppe erforderlich sind. Man kann daher eine Kante zwischen die zwei Knoten einfügen, um die zu einer Gruppe bilden zu lassen.

darf. Die Überprüfung, ob der Weg W sich innerhalb der Zeitgrenze befindet, ist in einer polynomiellen Zeit möglich.¹⁷

Somit lässt sich das Problem in einer nicht-polynomiellen Zeit lösen und die Lösungen können in einer polynomiellen Zeit überprüft werden, weshalb das Lieferkettenproblem in der Zusatzherausforderung eindeutig NP-Vollständig ist.

11 Zusatzherausforderung: Beispiele

Für die Beispiele wurden noch einige zusätzliche Beispiele erstellt, um Grenzszenarien darzustellen und um die Robustheit der Lösung zu beweisen. Alle Eingaben sowie Ausgaben sind im Ordner zu finden. Alle vorgegebenen Aufgaben stammen aus der Version vom 06.02.2026. Einige Standardbeispiele wurden aufgrund von sehr hohen Werten übersprungen.

11.1 Vorgegebene Aufgaben

Aufgabe	Ausgabe	Kommentar
lieferung00.txt	MOEGLICH 15 13 S [1 1] 2 [4 2] T 10 1 [2 1] 3 [5 2] T 15 4 T [5 2] T	Execution time: 357µs
lieferung01.txt	MOEGLICH 40 32 S [1 1] [4 2] [8 3] T 40 1 S [2 1] S 1 [4 2] [7 3] T 37 4 [5 2] [8 3] T 36 8 [6 3] T	Execution time: 555µs
lieferung02.txt	MOEGLICH 164 116 S [2 2] [13 5] 2 [4 3] 3 5 [6 4] [5 1] 6 1 T 130 2 13 7 5 [3 2] [5 1] [6 4] 5 [7 5] 13 2 [4 3] S 1 T 102 13 [7 5] 13 2 [4 3] 3 [5 1] [6 4] 1 T 164 4 3 5 6 12 [11 3] 12 [6 4] [1 1] T	Execution time: 2592µs

¹⁷Man müsste dafür wiederum den Graphen $G(V, E)$ mit $w : E \rightarrow \{1, \dots, 20\}$ erstellen, der alle Schritte nachsimuliert und die benötigte Länge aufsummiert. Am Ende erhält man eine Gesamtlänge, die man mit dem Limit vergleicht, um eindeutig die Lösung als gültig oder falsch zu bestimmen.

Aufgabe	Ausgabe	Kommentar
	116 6 [14 4] 6 [1 1] T 116 5 6 [1 1] T	
lieferung03.txt	MOEGLICH 232 154 S [2 9] [6 10] [4 1] 5 [19 2] [20 5] 22 [21 3] [22 6] 20 32 [31 4] 33 [28 8] [12 7] 26 36 T 196 2 S [3 9] [34 5] 3 S 2 [6 10] [4 1] 5 [19 2] 18 40 29 [27 3] [28 8] [12 7] 26 [15 4] [26 6] 36 T 210 6 2 32 31 [33 10] [31 4] 32 2 6 [4 1] 5 [19 2] [20 5] 19 18 40 29 [27 3] [28 8] [12 7] [26 6] 36 T 232 4 5 19 18 40 29 30 8 [7 1] [8 4] 30 29 40 18 [19 2] [20 5] 19 18 40 29 [27 3] [28 8] [12 7] [26 6] 36 T 184 19 5 16 [1 2] [16 3] 5 19 [20 5] 32 [31 4] 33 [28 8] [12 7] [26 6] 36 T 178 20 32 31 25 [34 5] [25 4] [23 3] [26 6] [12 7] [28 8] 12 26 36 T 172 21 22 20 19 18 40 29 [27 3] [28 8] [12 7] 26 [15 4] [26 6] 36 T 180 22 20 32 31 33 28 12 [26 6] [15 4] 26 [12 7] [28 8] 12 26 36 T 156 31 [32 4] 31 33 [28 8] [12 7] 26 36 T 190 28 12 26 37 [9 8] 37 26 [12 7] 26 36 T 194 12 26 37 [38 7] 37 26 36 T	Execution time: 521647µs
lieferung04.txt	MOEGLICH 175	Execution time: 29269914µs
lieferung09.txt	MOEGLICH 11	Execution time: 631µs

11.2 Eigene Aufgaben

Aufgabe	Ausgabe	Kommentar
bitmask_10w_100N.txt	MOEGLICH 34	Beispiel mit 10 Arbeitsschritte und 102 Knoten im Graphen. Execution time: 1273527 μ s
bitmask_10w_302N.txt	MOEGLICH 3297	Beispiel mit 10 Arbeitsschritte und 302 Knoten im Graphen. Execution time: 6917725 μ s
bitmask_15w_32N.txt	MOEGLICH 87	Beispiel mit 15 Arbeitsschritte und 32 Knoten im Graphen. Execution time: 26315255 μ s
bitmask_17w_36N.txt	MOEGLICH 89	Beispiel mit 17 Arbeitsschritte und 36 Knoten im Graphen. Execution time: 122008873 μ s Dieses Beispiel soll zeigen, dass die Lösung für eine angemessene Anzahl an Arbeitsschritten recht gut funktioniert. Für eine höhere Anzahl an Arbeitsschritten kann es (v.a. bei 20) zu Problemen aufgrund von hoher RAM-Auslastung führen.
bitmask_1w_2002N.txt	MOEGLICH 20975	Beispiel mit 1 Arbeitsschritt und 2002 Knoten im Graph. Execution time: 2539398 μ s
bitmask_1w_4002N.txt	MOEGLICH 41450	Beispiel mit 1 Arbeitsschritt und 4002 Knoten im Graph. Execution time: 10387339 μ s Für sehr viele Knoten ist diese Lösung ebenfalls nutzbar.
bitmask_5w_252N.txt	MOEGLICH 10 10 S [60 2] 250 [18 1] [161 4] 18 [147 3] [214 5] 239 219 T 10 60 250 98 [71 2] [161 4] [18 1] [147 3] [219 5] T 10 18 [12 1] 18 [161 4] 18 [147 3] [219 5] T 10 161 [182 4] [109 3] 239 [219 5] T	Beispiel mit 5 Arbeitsschritte und 252 Knoten im Graph. Execution time: 374589 μ s

Aufgabe	Ausgabe	Kommentar
	10 147 [105 3] 147 [219 5] T 10 214 239 [219 5] T Execution time: 374589 μ s	
bitmask_5w_502N.txt	MOEGLICH 9 9 S 392 [304 4] [34 1] [136 2] [475 5] 290 [268 3] 290 T 8 304 [362 4] [134 2] [43 1] [475 5] [290 3] T 9 34 136 [53 1] [136 2] [475 5] [290 3] T 9 136 [187 2] 136 [475 5] [290 3] T 9 475 [476 5] 475 [290 3] T 9 268 [290 3] T Execution time: 2561812 μ s	Beispiel mit 5 Arbeitsschritt und 502 Knoten im Graph. Execution time: 2561812 μ s
bitmask_c_100.txt	MOEGLICH 3893	Beispiel Gewichte mit einem Be- trag von bis zu 100 im Graph (Arbeitsschritte: 10, Knotenzahl: 202). Execution time: 3622513 μ s Lösung funktioniert auch mit höheren Gewichten. Bei sehr ho- hen Gewichten muss der Dial Algorithmus durch Dijkstra aus- getauscht werden.
bitmask_unreachable.txt	UNMOEGLICH Execution time: 81942 μ s	Es gibt keinen Weg vom Start- knoten S zum Endknoten T . Execution time: 81942 μ s Programm stürzt nicht ab.
bitmask_wis_1.txt	UNMOEGLICH Execution time: 396370 μ s	Für bestimmte Arbeitsschritte können nur ein Fabrik den Schritt durchführen Execution time: 396370 μ s

Aufgabe	Ausgabe	Kommentar
		Programm stürzt ebenfalls nicht ab.

12 Zusatzherausforderung: Quellcode

Aus Platzgründen wurden redundante Informationen verkürzt (gekennzeichnet durch [...]) oder weggelassen. Die wichtigsten Strukturen sind weiterhin vorhanden. Für den vollständigen Quellcode, sollte die im Abgabe-Ordner findbare Quelltext Datei angeschaut werden.

```
void solve() {
    int timelimit;
    cin >> timelimit;
    int tasks;
    cin >> tasks;
    [...] // Redundanz
    int w;
    cin >> w;
    int N = w + 2;
    vector<vector<int>>> ids(tasks + 2); // Arbeitsschritte -> Fabriken
    vector<int> rev_ids(N); // Fabriken -> Arbeitsschritte
    ids[0].push_back(0); // Startknoten
    rev_ids[0] = 0;
    ids[tasks + 1].push_back(N - 1); // Endknoten
    rev_ids[N - 1] = tasks + 1;
    [...] // Einlesen der Daten für ids sowie rev_ids

    int M;
    cin >> M;
    vector<vector<pair<int, int>>> adj(N);
    int maxWeight = 0;
    for (int i = 0; i < M; i++) {
        [...] // Einlesen der Kanten für Graph G
    }

    // dial's algorithm (Code abgeändert aus https://www.geeksforgeeks.org/dsa/dial's-
    algorithm-optimized-dijkstra-for-small-range-weights/)

    vector<vector<int>>> backtrack(N, vector<int>(N, -1));
    vector<vector<int>>> dist(N, vector<int>(N, 1e9));
    int maxDist = (N - 1) * maxWeight;
    vector<vector<int>>> buckets(maxDist + 1);

    for (int i = 0; i < N; i++) {
        dist[i][i] = 0;
        backtrack[i][i] = i;
        buckets[0].push_back(i);
        int highDist = 0;
        for (int d = 0; d <= maxDist && d <= highDist; d++) {
            while (!buckets[d].empty()) {
                int u = buckets[d].back();
                buckets[d].pop_back();

                if (d > dist[i][u]) continue;
                for (auto& [v, weight] : adj[u]) {
                    int newDist = dist[i][u] + weight;

                    if (newDist < dist[i][v]) {
                        dist[i][v] = newDist;
                    }
                }
            }
            highDist = d;
        }
    }
}
```

```

        backtrack[i][v] = u;
        buckets[newDist].push_back(v);
        highDist = max(highDist, newDist);
    }
}
}

// Für jedes Werkstatt ein alternativwerkstatt
vector<vector<int>> backups(1 << (tasks + 1), vector<int>(N, -1));
// kürzester Weg von aktuelle Knoten zur Endknoten (unter Beachtung von Werkstätten, die
noch besucht werden müssen)
vector<vector<int>> defaultPaths(1 << (tasks + 1), vector<int>(N, -1));
// Der beste Weg für den fall, dass keine Werke streiken.
vector<vector<int>> recommendedPaths(1 << (tasks + 1), vector<int>(N, -1));

vector<vector<vector<int>>> DP(1 << (tasks + 1), vector<vector<int>>(N, vector<int>(4,
1e9)));

// Zielknoten -> 0 Distanz
[...] // Alle DP-Werte der Zielknoten wird auf 0 gesetzt.

// Kürzester Weg vom Endknoten berechnen
for (int mask = 0; mask < (1 << (tasks)); mask++) {
    for (int task = 0; task <= tasks; task++) {
        if (mask & (1 << task)) continue;
        int counter = mask;
        while (counter) {
            int startWorkstation = LOG2(counter & -counter) + 1; // Letzte besuchte Werk
            counter -= counter & -counter;
            for (int u : ids[startWorkstation]) {
                for (int v : ids[task + 1]) { // Alle Werke mit noch nicht durchgeführte
Arbeitsschritte besuchen.
                    if (DP[mask][u][0] + dist[u][v] < DP[mask | (1 << task)][v][0]) {
                        DP[mask | (1 << task)][v][0] = DP[mask][u][0] + dist[u][v];
                        defaultPaths[mask | (1 << task)][v] = u;
                    }
                }
            }
        }
    }
}

// Kürzester Umweg zu einer alternativwerkstatt bestimmen
for (int task = 0; task <= tasks; task++) {
    if (!(mask & (1 << task))) continue;
    for (int u : ids[task + 1]) {
        for (int v : ids[task + 1]) {
            if (u == v) continue;
            if (DP[mask][v][0] + dist[u][v] < DP[mask][u][1]) {
                DP[mask][u][1] = DP[mask][v][0] + dist[u][v];
                backups[mask][u] = v;
            }
        }
    }
}

// Empfohlener Weg bestimmen
for (int mask = 0; mask < (1 << tasks); mask++) {
    // Beim erste mal durchführen: Alle mögliche Wege aus der Endknoten berechnen.

```

```

    if (mask == 0) {
        for (int task = 0; task <= tasks; task++) {
            for (int u : ids[task + 1]) {
                for (int v : ids[tasks + 1]) {
                    if (DP[mask][v][2] + dist[v][u] < DP[mask | (1 << task)][u][2]) {
                        DP[mask | (1 << task)][u][2] = DP[mask][v][2] + dist[v][u];
                        DP[mask | (1 << task)][u][3] = DP[mask][v][3] + dist[v][u];
                        recommendedPaths[mask][u] = v;
                    }
                }
            }
            // Wenn der aktuelle Alternativwerk länger dauert als alle zuvorige
Alternativwerke
            if (DP[mask | (1 << task)][u][2] < DP[mask | (1 << task)][u][1]) {
                DP[mask | (1 << task)][u][2] = DP[mask | (1 << task)][u][1];
            }
        }
    }

    // Wenn der aktuelle Alternativwerk länger dauert als alle zuvorige Alternativwerke
    {
        int counter = mask;
        while (counter) {
            int startWorkstation = LOG2(counter & -counter) + 1;
            counter -= counter & -counter;
            for (int u : ids[startWorkstation]) {
                DP[mask][u][2] = max(DP[mask][u][2], DP[mask][u][1]);
            }
        }
    }

    // aus alle besuchte Werke -> Nächste Werke, dessen Arbeitsschritt noch nicht
    durchgeführt wurde.
    for (int task = 0; task <= tasks; task++) {
        if (mask & (1 << task)) continue;
        int counter = mask;
        while (counter) {
            int startWorkstation = LOG2(counter & -counter) + 1;
            counter -= counter & -counter;
            for (int u : ids[task + 1]) {
                for (int v : ids[startWorkstation]) {
                    if (DP[mask][v][2] + dist[v][u] < DP[mask | (1 << task)][u][2]) {
                        DP[mask | (1 << task)][u][2] = DP[mask][v][2] + dist[v][u];
                        DP[mask | (1 << task)][u][3] = DP[mask][v][3] + dist[v][u];
                        recommendedPaths[mask][u] = v;
                    }
                }
            }
        }
    }

    // Aus alle Werke (unter der Voraussetzung, dass alle Arbeitsschritte durchgeführt wurden)
    den kürzesten Weg zur Startknote berechnen.
    int mask = (1 << (tasks)) - 1;
    for (int task = 0; task <= tasks; task++) {
        for (int u : ids[task + 1]) {
            if (DP[mask][u][2] + dist[u][0] < DP[mask][0][2]) {
                DP[mask][0][2] = DP[mask][u][2] + dist[u][0];
                DP[mask][0][3] = DP[mask][u][3] + dist[u][0];
                recommendedPaths[mask][0] = u;
            }
        }
    }

```

```

    }
}

// Falls das Betrag der worst case Streckenlänge das Zeitlimit überschreiten sollte. ->
Kein Weg möglich
if (DP[(1 << (tasks)) - 1][0][2] > min(timelimit, (int) 1e9 - 1)) {
    cout << "UNMOEGLICH" << endl;
    return;
}
cout << "MOEGLICH" << "\n";
cout << DP[mask][0][2] << "\n";
cout << "\n";
cout << DP[mask][0][3] << "\n";
getTrack(recommendedPaths, backtrack, rev_ids, mask, 0, N); // Empfohlener Strecke
ausgeben
int countmask = mask;
int pos = 0;
// Alle Fällenbearbeitung, falls einzelne Werke geschlossen sind.
while (countmask) {
    pos = recommendedPaths[countmask][pos];
    cout << DP[mask][0][3] - DP[countmask][pos][3] + DP[countmask][pos][1] << "\n"; //
Gesamtlänge der Alternativstrecke berechnen
    getBackupTrack(defaultPaths, backtrack, rev_ids, countmask, pos, N, backups[countmask]
[pos]); // Alternativstrecke ausgeben.
    countmask = countmask & ~(1 << (rev_ids[pos] - 1)); // Arbeitsschritt als fertig
markieren.
}
cout << flush;
}

```